Budapesti Műszaki SZC Bláthy Ottó Titusz Informatikai Technikum

1032 Budapest, Bécsi út 134.

Szoftverfejlesztő- és tesztelő képzés

VIZSGAREMEK

Porsche Sonderwunsch autókonfigurátor webalkalmazás
fejlesztése Laravelben, Vue.js JavaScript keretrendszer alkalmazásával

Készítette:

Andorkó Zsolt

Varga Olivér

2026

Tartalomjegyzék

1 A projekt ismertetése 3
2 Felhasználói dokumentáció 3
2.1 Felületek ismertetése 3
2.2 Funkciók ismertetése és használati instrukciók 9
3 Fejlesztői dokumentáció 10
3.1 Munkamegosztás 10
3.1.1 Szerepkörök és felelősségi körök 10
3.2 Telepítési dokumentáció 12
3.3 Adatbázis dokumentáció 13
3.3.1 Diagram 13
3.3.2 Mezők részletes ismertetése 14
3.3.3 Kapcsolatok indoklása, ismertetése 15
3.4 Komponens/struktúra dokumentáció 16
3.4.1 A teljes rendszer felépítése 16
3.4.2 A projekt elemei fejlesztés szempontjából 16
3.4.3 Osztályok 16
3.4.4 Kapcsolatok 17
3.5 API dokumentáció 17
3.6 Teszt dokumentáció 25
4 Továbbfejlesztési lehetőségek 26
5 Összegzés 27
6 Források ..... **Error! Bookmark not defined.**

1 A projekt ismertetése

A Porsche Sonderwunsch autókonfigurátor a Szoftverfejlesztő- és tesztelő képzés vizsgafeladataként készült, és a képzés során elsajátított tudást hivatott bemutatni egy közös munka keretében. A projekt egy modern, webalapú alkalmazás, amely lehetővé teszi a felhasználók számára, hogy különböző Porsche modelleket válogassanak össze, testre szabják azok színét, kerekeit, belső kialakítását és extra tartozékait.

Az alkalmazás célja egy életszerű autóértékesítési folyamat digitális támogatása, az ügyfél igényeinek rögzítésétől a rendelés leadásáig. A felhasználó három klasszikus modell a GT3 997.2, a Carrera GT és a 930 Turbo közül választhat, majd részletesen konfigurálhatja az autót az adott modellhez tartozó szín-, felni-, belső- és kiegészítő opciókkal. A rendszer minden választásnál valós időben frissíti az árat, így a felhasználó azonnali visszajelzést kap a konfiguráció pénzügyi hatásáról.

A program a felhasználó által összeállított konfigurációt menti, majd ebből rendelést is leadhat. A rendelést a rendszer „pending" állapottal hozza létre, és csak admin (manager) jogosultságú felhasználó tudja módosítani vagy „completed" / „cancelled" állapotúra állítani. A normál felhasználó (user) saját profiljához fér hozzá: itt látja és szerkesztheti a saját adatait, megnézheti a mentett konfigurációit és a leadott rendeléseit. Az admin felhasználó ezzel szemben az összes regisztrált felhasználó és rendelés adatait is eléri, és módosíthatja azokat.

A feladat megvalósításához backend oldalon a Laravel 12 nyílt forráskódú PHP keretrendszert alkalmaztuk MySQL relációs adatbázis-kezelő rendszer használatával, valamint Laravel Sanctum token-alapú API hitelesítéssel. A frontend Vue.js 3 JavaScript keretrendszerben, Bootstrap 5 stílussal készült, az API hívásokat Axios kliens kezeli, az útvonalkezelést pedig a Vue Router. A rendszer architektúrája tisztán szétválasztja a kliens- és szerveroldalt: a két réteg között kizárólag REST API hívások történnek, JSON formátumban.

2 Felhasználói dokumentáció

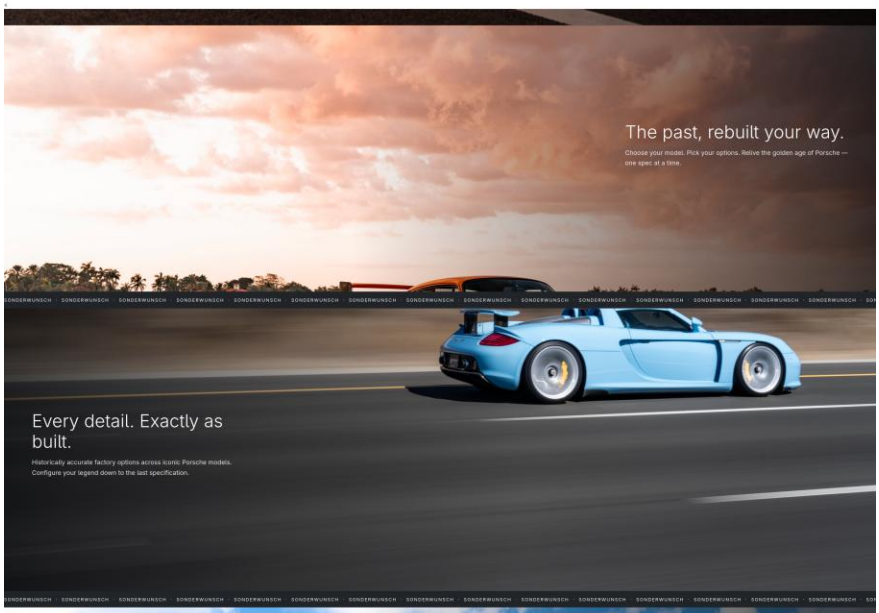
2.1 Felületek ismertetése

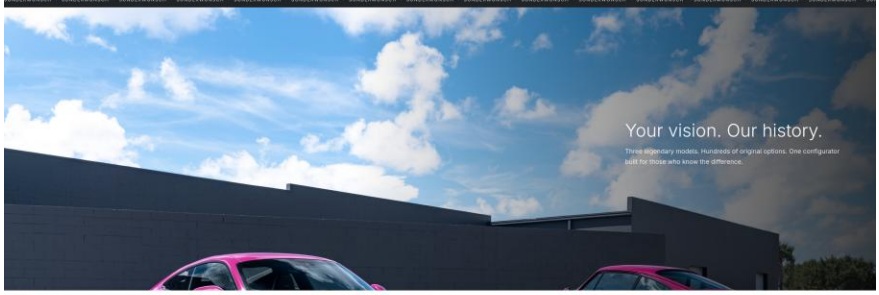
A Porsche Sonderwunsch alkalmazás belépés nélkül is böngészhető: a főoldal (Home View) és a modellek nyilvánosan elérhetők, a konfigurátor és a rendelés leadása azonban már bejelentkezést igényel. A rendszer három különböző layoutot használ: a nyilvános oldalakat a PublicLayout (NavBar + tartalom + FooterBar), a bejelentkezési és regisztrációs felületeket az AuthLayout, míg a védett, bejelentkezés után elérhető oldalakat az AppLayout keretezi.

A főoldal (Home View) marketing jellegű: egy carousel a kiemelt modellekkel és három-három képpel illusztrált „Sonderwunsch" hero-szekció vezeti be a látogatót a programba. Az oldal alján egy lépésekre bontott magyarázat jelenik meg arról, hogy a felhasználónak milyen folyamaton kell végigmennie ahhoz, hogy egy egyedi konfigurációhoz jusson.

Sonderwunsch – Your Dream, Our Craft

The Porsche Sonderwunsch programme allows every car to be truly one of a kind. Choose your model, personalise it down to the finest detail, and create the car you've always dreamed of. Whether it's a classic 911 or the modern Taycan – we turn every vision into





Your path to your Porsche

You Choose the Car

Browse our selection of iconic Porsche models and pick the one that speaks to you. Filter by model, era, and specification — we'll help you find the perfect match.

We Order & Receive It

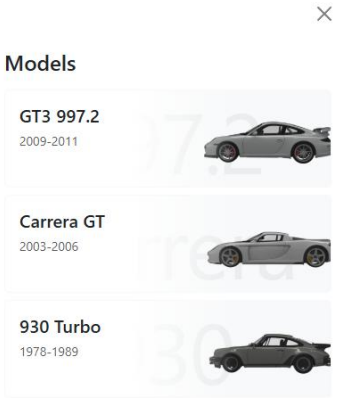
Once you've made your choice, we handle the rest. We source, purchase, and have the car delivered directly to our facility — no hassle, no compromise.

We Build Your Vision

Our team gets to work. Using your exact configuration and historically accurate parts, we build your Porsche from the ground up — exactly the way you specified it.

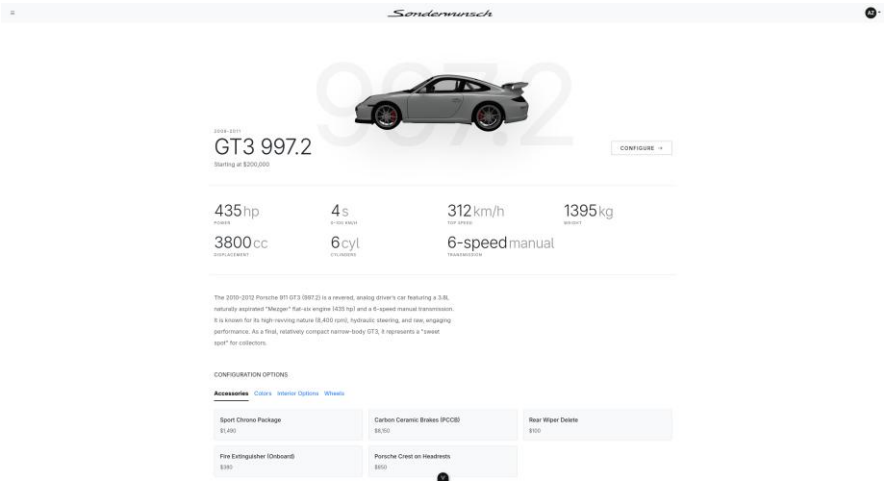
1. ábra – Home View – főoldal

A főoldalról (1. ábra) a felhasználó a navigációs sáv bal oldalán található listaikonra kattintva oldalsávot (offcanvas) nyithat meg (2. ábra), amelyben az elérhető modellek ABC-rendben kártyák formájában jelennek meg. A kártyák hover-effekttel kiemelődnek, és a háttérben halványan szerepel a modell neve egy nagy tipográfiai elemként. Bármelyik kártyára kattintva a felhasználó a modellnézetre (Model View) jut.



2. ábra – NavBar – modell választó

A modellnézet (Model View) tartalmazza a kiválasztott autó hero-képét, a kezdő árát, a műszaki adatokat (teljesítmény, gyorsulás, végsebesség, súly, hengerűrtartalom, hengerszám, sebességváltó), majd egy hosszabb leírást a modellről. Ez alatt egy fülrendszer (tabok) található: accessories, colors, interior_options, wheels, ahol a felhasználó megtekintheti a választható opciókat.



3. ábra – Model View – részletes modelloldal

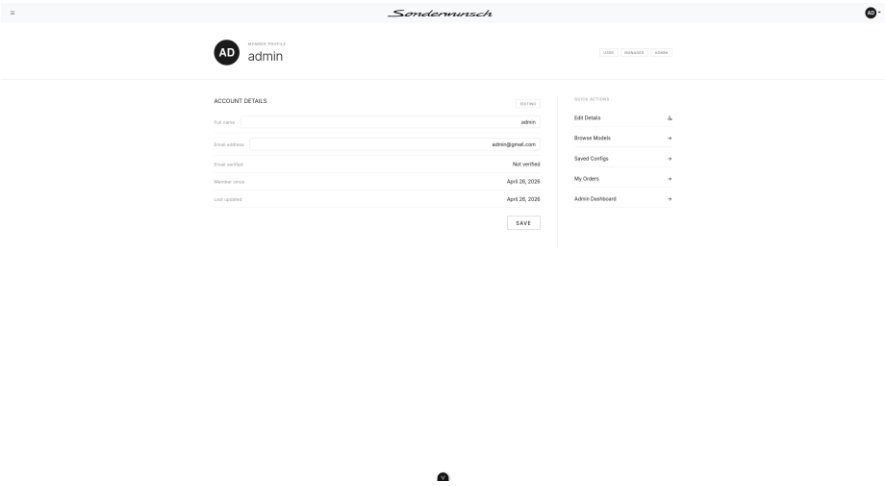
A modelloldal (3. ábra) felső részén jobb oldalt található a „Configure" gomb, amely átviszi a felhasználót a konfigurátor felületre. Ehhez bejelentkezés szükséges, ezért a router meta requiresAuth jelzéssel védi az útvonalat. A konfigurátor két paneles felépítésű: a bal oldali nagy

területen kerül majd a 3D autómodell megjelenítésére (jelenleg placeholder), míg a jobb oldali sáv egy összecsukható accordion-t tartalmaz az opciókhoz Color, Wheels, Interior, Accessories – és felül az aktuális összárral.

4. ábra – Configure View – konfigurátor

A konfigurátor (4. ábra) működése úgy van kialakítva, hogy a felhasználó egy szín- és felni választást azonnal megfigyelhet az árban, az interior és accessories pedig pivot táblákon keresztül több elemmel is választhatóak. A jobb alsó sarokban két gomb található: „Save Configuration" (mentés a profilba) és „Place Order" (azonnali rendelés leadása). Az utóbbi a háttérben automatikusan menti a konfigurációt is, mielőtt rendelést hozna létre belőle.

A bejelentkezett felhasználó számára elérhető a profil (Profile View). Hero-szerű felső sávval rendelkezik, ahol a generált avatar, a felhasználó neve és az aktuális jogosultságai (user, manager, admin) láthatók. Alatta osztott elrendezés található: a bal oldali nagyobb panel a kiválasztott aloldalt jeleníti meg (RouterView), a jobb oldali keskenyebb panel a gyors navigációs linkeket mutatja: Profile Details, Browse Models, Saved Configs, My Orders és admin esetén Admin Dashboard.

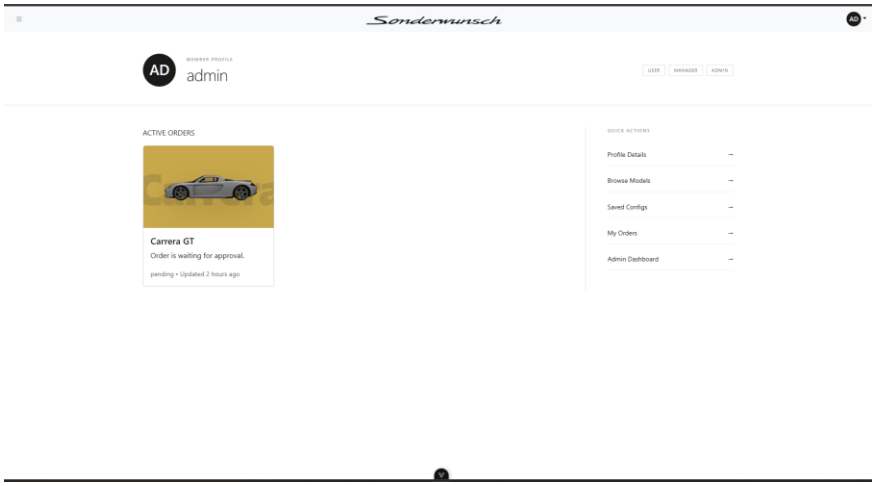


5. ábra – Profile Details View

A Profile Details (5. ábra) megjeleníti a felhasználó alapadatait: név, email, verifikációs állapot, regisztráció dátuma és utolsó frissítés. A jobb felső sarokban található „Edit Details"

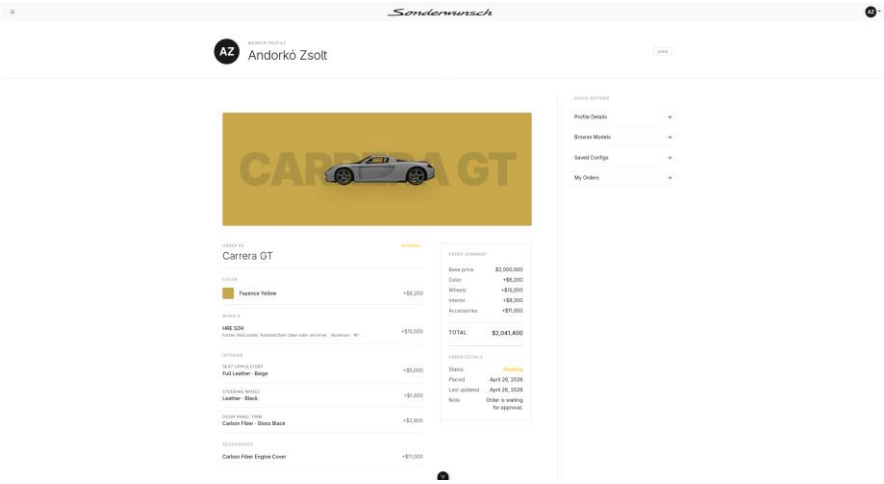
linkkel a felület inline szerkesztési módba kapcsol, ahol a név, email és jelszó módosítható; a backend pedig azonnal érvényesíti a változtatásokat (validációs hibák a mezők alatt jelennek meg).

A „My Orders" oldalon (OrderListView) a felhasználó saját rendelései egy carousel komponensben jelennek meg, kártyák formájában. Minden kártyán látható a kiválasztott modell, az aktuális státusz (pending/completed/cancelled), a felhasználó által megadott üzenet és a frissítés időpontja relatív formában (pl. „2 hours ago"). A kártyára kattintva a részletes nézetre jut.



6. ábra – OrderListView – saját rendelések

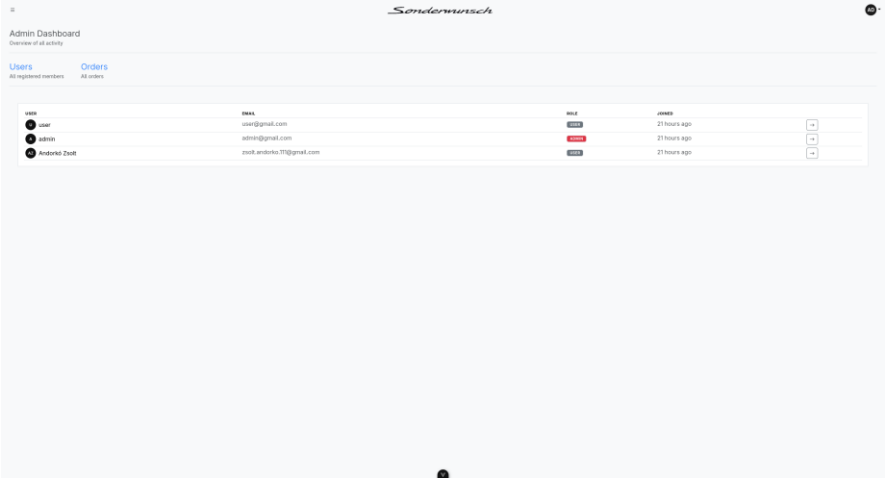
A részletes rendelésnézet (OrderDetailsView) (6. ábra utáni felület) bemutatja a rendeléshez tartozó konfiguráció minden részletét: szín a hex kódjával, kerék (név, anyag, méret, szín), belső opciók partok szerint csoportosítva, valamint kiválasztott kiegészítők. Jobb oldalon egy összesítő panel jeleníti meg a teljes árbontást és a rendelés metaadatait.



7. ábra – OrderDetailsView – rendelés részletei

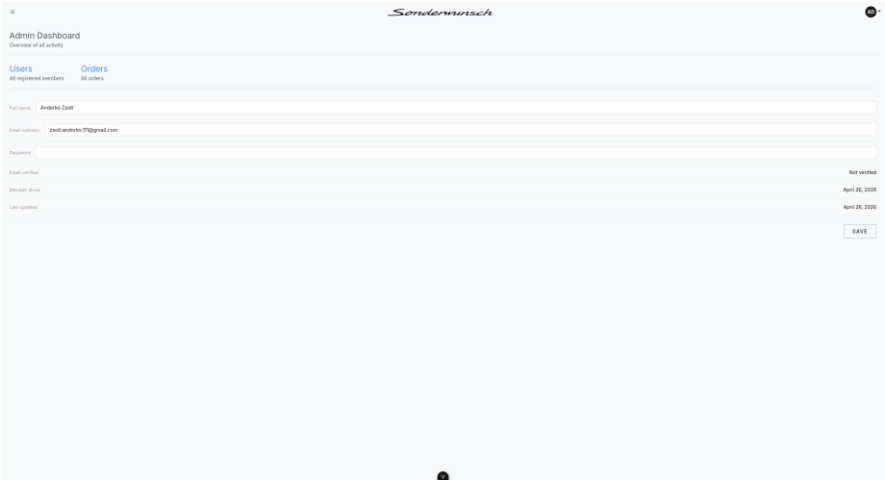
A részletek nézet (7. ábra) középpontjában egy nagy hero kép áll a választott színből mintaszerűen vett háttérrel, ami vizuálisan is összeköti a felhasználó által konfigurált autót a megjelenítéssel. Ez a kártyás bontás teszi áttekinthetővé az egyébként összetett konfigurációs adatot.

Az adminisztrációs felület külön csoportot képez a routing-ban: a /app/admin/* útvonalak alatt érhető el, és csak admin abilities-zel rendelkező token tudja megnyitni (a router beforeEach guard 403-as oldalra navigál egyébként). Az Admin Dashboard tartalmaz egy mellékmenüt és három fő nézetet: felhasználók listája, rendelések listája és a hozzájuk tartozó részletes szerkesztők.



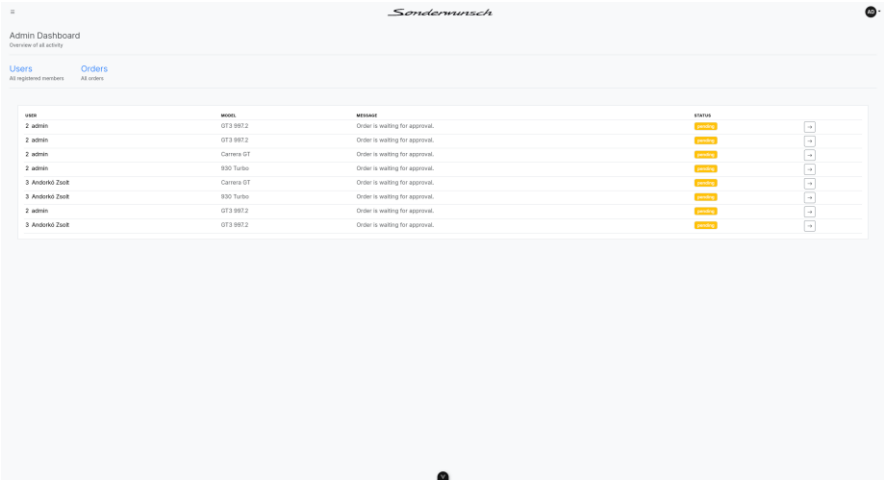
8. ábra – Admin – felhasználók listája

Az admin felhasználói listán (8. ábra) a felhasználók táblázatos formában jelennek meg, ID, név, email és szerepkör (role) adatokkal. Egy felhasználóra kattintva a részletes szerkesztő oldalra (AdminUserDetailsView) jut, ahol a manager/admin token birtokában a név, email és jelszó módosítható.



9. ábra – Admin – felhasználó szerkesztése

A felhasználó-szerkesztő (9. ábra) inputmezőkben jeleníti meg a meglévő értékeket, és a „Save" gombbal elküldi a módosítást a /api/auth/update/{{id}} végpontnak. Ha hibás adat érkezik (pl. már foglalt email vagy túl rövid jelszó), a backend 422-es válasszal és a hibás mezők alá beillesztett hibaszöveggel reagál.



10. ábra – Admin – rendelések listája

Az admin rendelési listanézetben (10. ábra) az összes felhasználó rendelése látható egy közös táblázatban, a felhasználóhoz tartozó modell, üzenet és státusz oszlopokkal. Egy sor kiválasztásakor a manager az AdminOrderDetailsView nézetben tudja módosítani a státuszt (pending → completed/cancelled) és üzenetet írhat hozzá. Ez a felület közvetlenül kommunikál a /api/orders/{{id}} PATCH végponttal.

2.2 Funkciók ismertetése és használati instrukciók

A bejelentkezési felület egy normál, e-mail – jelszó kombinációt kérő formot tartalmaz, a navigációs sáv a hitelesítési layoutban (AuthLayout) elrejtésre kerül. A bejelentkezést a frontend a /api/auth/login végponton hívja meg, és sikeres válasz esetén a token, a felhasználói adatok és a token abilities a localStorage-ba kerül. Az App.vue gyökérkomponens a betöltés után automatikusan meghívja az /api/auth/check végpontot, hogy a token érvényességét megerősítse.

A bejelentkezéshez a teszt felhasználó-fiókok használhatók:

- user@gmail.com / User1234 → normál felhasználó (user szerepkör)
- admin@gmail.com / Admin1234 → adminisztrátor (admin szerepkör)

A regisztrációs oldalon (RegisterView) a felhasználó név, email és jelszó megadásával hozhat létre új fiókot. A backend RegisterAuthRequest validációs osztálya ellenőrzi: a név kötelező és maximum 255 karakter; az email kötelező, valid formátumú és egyedi; a jelszó kötelező, minimum 8 karakter. Sikeres regisztráció után a felhasználó a bejelentkezési oldalra kerül.

Modellek böngészésekor a felhasználó vagy a navigációs sáv offcanvas listájából, vagy a főoldal carousel-éből választhat egyet. A modelloldalra érkezve a frontend /api/models/{id} végpontot hívja, amely visszaadja a modell alapadatait, a műszaki specifikációt és az opciókat (colors, wheels, interior_options, accessories).

A „Configure" gombra kattintva a felhasználó a konfigurátor oldalra navigál. A konfigurálás során négy döntés érhető el: szín (Color - egy darab), kerék (Wheels - egy darab), belső (Interior – partok szerint csoportosítva, partonként legfeljebb egy elem), kiegészítők (Accessories – tetszőleges számú, többszörös választás). A totalPrice computed property minden választást aggregál. A „Save Configuration" gomb a /api/configs POST végpontot hívja, és átirányít a saját

konfigurációk listájára. A „Place Order" gomb először létrehozza a konfigurációt, majd a /api/orders POST végpontnak küldi az új rendelést „pending" státusszal.

A profil oldalon a felhasználó megtekintheti és szerkesztheti az adatait, valamint a jobb oldali gyorsmenüből navigálhat a saját mentett konfigurációi és rendelései listájához. Az „Edit Details" linkre kattintva a profil részleteinek nézete inline szerkesztési módba kapcsol, és a „Save" gombbal a /api/auth/update/{userId} végpontnak küldi a módosításokat.

Adminisztrátorként a felhasználó a profilból a „Admin Dashboard" link megnyomásával jut a vezérlőpultra. Itt két fő funkció érhető el: felhasználók kezelése és rendelések kezelése. A táblázatban való kiválasztás után a manager módosíthatja a felhasználói adatokat vagy a rendelés állapotát; ezek a műveletek a backend manager/admin abilities-szel védett végpontjait hívják.

A navigációs sáv jobb oldalán található dropdown-ben a „Sign out" linkre kattintva a felhasználó kijelentkezhet. Ez egyrészt a /api/auth/logout POST végpontot hívja a backenden a token törléséhez, másrészt törli a localStorage-ból a tárolt adatokat, majd átirányítja a felhasználót a főoldalra.

3 Fejlesztői dokumentáció

3.1 Munkamegosztás

A Porsche Sonderwunsch projekt fejlesztése kétfős csapatban, Git verziókövetéssel és pull request alapú munkamenettel valósult meg. Összesen 21 pull request került mergelésre a fő fejlesztési ciklus alatt, mindegyik review és Approved jelzés mellett. A munkamegosztás a projekt kezdetétől fogva nem szigorúan backend-frontend felosztású volt: mindketten dolgoztunk mindkét rétegen, a feladatok inkább funkcionális egységenként (autentikáció, modellnézet, konfigurátor, admin felület stb.) lettek elosztva. Ez lehetővé tette, hogy mindketten teljes körű ismeretet szerezzünk a teljes stackről.

3.1.1 Szerepkörök és felelősségi körök

Andorkó Zsolt (a-zsolt)

Andorkó Zsolt vitte a projekt repository tulajdonosi szerepét, és ő végezte a kezdeti backend alapozást, valamint a frontend főbb funkcionális blokkjainak megvalósítását. Az ő pull requestjei (#1, #2, #3, #4, #8, #9, #14, #17, #19, #21) felelősek a következőkért:

- Feat/public views (#1) – a publikus oldalak alapszerkezete, layoutok, a Home és Model nézetek első verziói.
- feat: basic authentication and full polish (#2) – Sanctum token-alapú bejelentkezés, regisztráció, kijelentkezés és a hozzájuk tartozó frontend nézetek.
- feat(frontend): model UI – cards, offcanvas menu & model detail view (#3) – modellkártyák, oldalsó offcanvas menü és a részletes modelloldal.
- Added validation to forms (#4) – űrlap-validációk a regisztrációs és bejelentkezési felületeken, hibák megjelenítése.
- Added profile view and per user privileges (#8) – a profiloldal kialakítása, jogosultsághoz kötött navigáció.

- Enable users and admins to edit profile details (#9) – a profil részletek szerkesztésének (név, email, jelszó) backend és frontend támogatása, admin oldali user-update végpont.
- Fix backend response and seeder; add initial config view (#14) – backend válaszok egységesítése, seeder javítások, konfigurátor első verziója.
- Allow saving configuration without interior and accessory options (#17) – a konfiguráció mentése pivot tábláknál optionális mezőkkel, validáció lazítása.
- Implement multi-option support and update order details UI (#19) – több belső opció és kiegészítő egyszerre választása, rendelés-részletek UI.
- Add comments to remaining files (#21) – kódkommentek hozzáadása a fennmaradó fájlokban.

Varga Olivér (oliver01010101)

Varga Olivér felelt a frontend csiszolásáért, az admin dashboardért és számos backend-frontend integrációs javításért. Az ő pull requestjei (#5, #6, #7, #10, #11, #12, #13, #15, #16, #18, #20) felelősek a következőkért:

- Add FooterBar component and complete HomeView updates (#5) – FooterBar komponens, a HomeView végleges szekciói és stílusa.
- Add discover button and footer scroll button in HomeView (#6) – discover CTA gomb és „scroll to top" navigációs elem.
- Backend fixes, Frontend ModelView and ModelCard (#7) – backend válaszformátum-javítások és a ModelView, ModelCard véglegesítése.
- fix(backend): allData fetch, fixed routes (#10) – /api/auth/allData manager-védett végpont kialakítása, route hibák javítása.
- Fix data fetching and add admin dashboard with UserSeeder (#11) – admin dashboard nézet, UserSeeder a teszt admin/user fiókokhoz.
- Create DetailsView and fix related issues (#12) – részletes adatnézetek (rendelés, konfiguráció) és kapcsolódó frontend hibák.
- Fix bugs and add NotAuthorizedView and NotFoundView (#13) – 403 és 404 hibaoldalak, kapcsolódó router-szabályok.
- Fix and update frontend views (#15) – frontend nézetek finomhangolása, reszponzivitás.
- Add wheel and color options with image URL to Wheel_Options (#16) – kerék- és színopciók kibővítése, kép URL mező a wheel__options táblához.
- Fix image rendering issues in frontend (#18) – képek megjelenítési problémáinak javítása (CDN URL-ek, méretezés).
- feat: add frontend and backend comments (#20) – kódkommentek a frontend és backend fájlokban.

Közös feladatok és egyeztetések

A projekt teljes ideje alatt rendszeresen egyeztettünk a mezőnevekről, a JSON válaszok struktúrájáról és az adatbázis felépítéséről. Mindkét fejlesztő dolgozott backend és frontend kódon is: Andorkó Zsolt elsősorban az autentikáció, profilkezelés és a konfigurátor üzleti logikájáért, Varga Olivér pedig a frontend nézetek polírozásáért, az admin dashboardért és a tesztadatok beállításáért felelt elsődlegesen. A pull request alapú workflow, a kódáttekintések (Approved jelzéssel) és a tagonkénti commitok közös működtetése a projektmenedzseri készségek fejlesztésére is jó alkalom volt.

3.2 Telepítési dokumentáció

A vizsgaremek két különálló projektből épül fel: egy Laravel 12 alapú backendből és egy Vue 3 alapú frontendből. A rendszer telepítése és futtatása mindkét rész megfelelő előkészítését igényli.

Backend (Laravel 12)

A backend a Laravel 12 keretrendszerre épül, MySQL adatbázissal és Laravel Sanctum 4.0 API-hitelesítéssel. A fejlesztéshez PHP 8.2 vagy újabb verzió, Composer, valamint egy kompatibilis adatbázis-kezelő szükséges. A projekt telepítése a függőségek letöltésével kezdődik, majd a .env környezeti fájl létrehozásával és az alkalmazáskulcs generálásával folytatódik. Telepítési lépések:

Lépés	Parancs
Függőségek telepítése	composer install
Környezeti fájl létrehozása	cp .env.example .env
Alkalmazáskulcs generálása	php artisan key:generate
Adatbázis kapcsolat beállítása	.env: DB_DATABASE, DB_USERNAME, DB_PASSWORD
Adatbázis inicializálása	php artisan migrate --seed
Fejlesztői szerver indítása	php artisan serve

Frontend (Vue 3 + Vite)

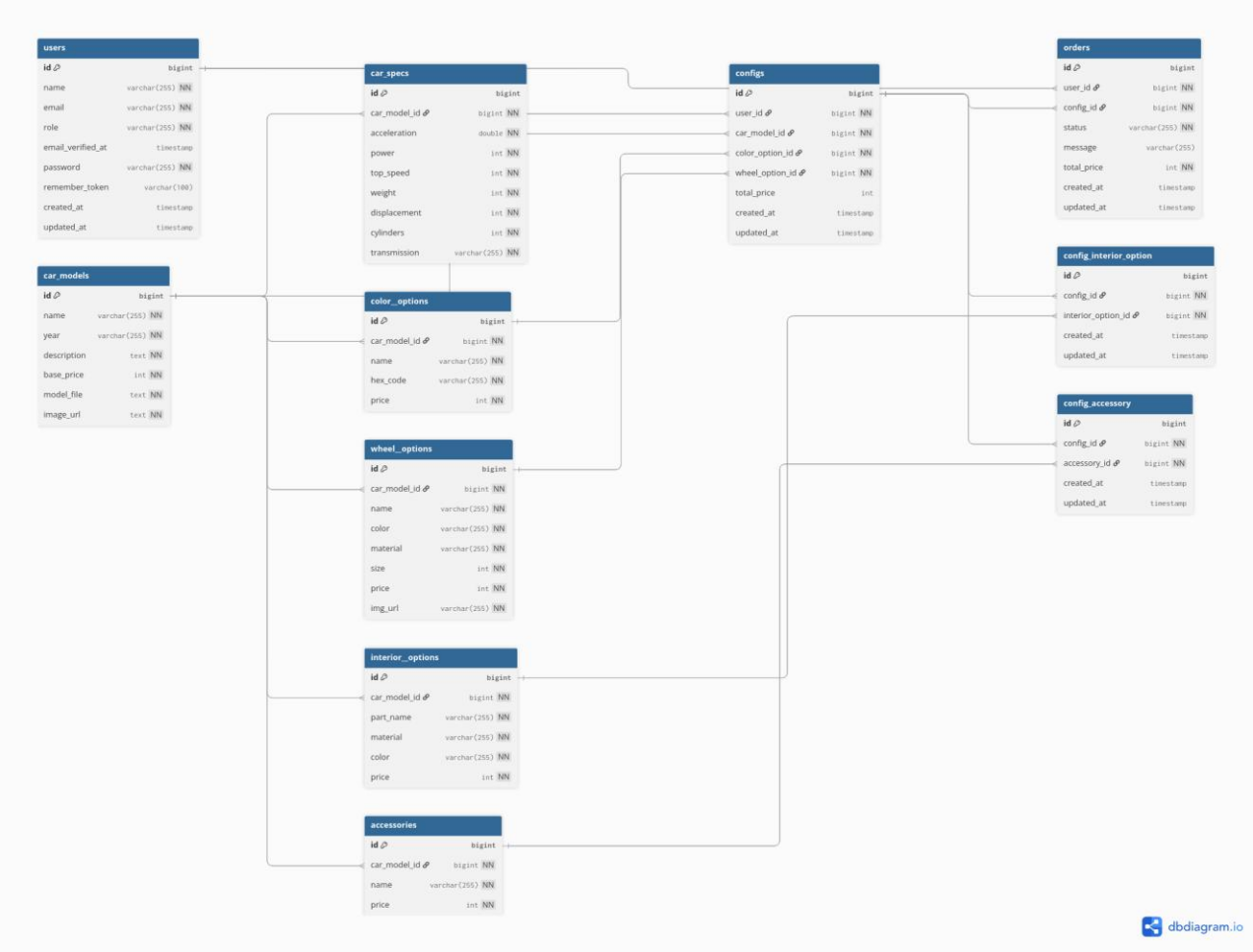
A frontend modern Vue 3 (Options API) környezetben készült, Vite 7-tel. A fejlesztéshez Node.js 20.19 vagy újabb és npm szükséges. A kliens alapértelmezett API címe: http://localhost:8000/, ezt a Frontend/src/utils/http.js fájlban a baseURL beállítás tartalmazza.

Lépés	Parancs
Függőségek telepítése	npm install
Fejlesztői szerver indítása	npm run dev
Production build	npm run build
Build előnézet	npm run preview

A telepítés után a backend a 8000-es porton, a frontend pedig a Vite alapértelmezett 5173-as portján fut. A két komponens párhuzamosan futtatható, és a frontend Axios kliense automatikusan a backend címére küldi a hívásokat.

3.3 Adatbázis dokumentáció

A rendszer adatbázisa relációs MySQL adatbázis, amely tizenegy fő táblára épül: users, car_models, car_specs, color__options, wheel__options, interior__options, accessories, configs, config_interior_option, config_accessory és orders. Ezek együtt biztosítják az autómodellek tárolását, a konfigurációs opciók kezelését, a felhasználók és rendelések nyilvántartását, valamint a több-a-többhöz kapcsolatokat.



11. ábra – Adatbázis diagram

3.3.1 Diagram

A car_models tábla a központi entitás, amelyhez 1:1 kapcsolatban (hasOne) tartozik a car_specs (műszaki adatok), 1:N kapcsolatban (hasMany) pedig négy opciótábla: color__options, wheel__options, interior__options és accessories. A users tábla 1:N kapcsolatban áll a configs táblával és az orders táblával. A configs tábla központi szerepet játszik a konfigurációs logikában: egy konfiguráció egy felhasználóhoz, egy modellhez, egy színhez és egy felnihez tartozik (belongsTo), valamint több belső opcióhoz és kiegészítőhöz kapcsolódhat egy-egy pivot táblán

keresztül. Az orders tábla a configs alapján jön létre, és belőle automatikusan számolódik a total_price.

3.3.2 Mezők részletes ismertetése

Users tábla:

Mező	Típus	Leírás
id	bigint, PK	Egyedi felhasználói azonosító
name	string(255)	Felhasználó neve
email	string(255), unique	Bejelentkezési e-mail cím
role	string default 'user'	Szerepkör: user / manager / admin
email_verified_at	timestamp, nullable	Email-verifikáció dátuma
password	string	Hashelt jelszó (bcrypt)
remember_token	string	Emlékeztető token
timestamps	created_at, updated_at	Laravel automatikus mezők

Car_models tábla:

Mező	Típus	Leírás
id	bigint, PK	Egyedi modell azonosító
name	string(255), unique	Modell neve (pl. „GT3 997.2")
year	string(255)	Gyártási évek (pl. „2009-2011")
description	text	Részletes modell leírás
base_price	integer	Alapár USD-ben
model_file	text	3D modell fájl elérési útja
image_url	text	Kép URL-je

Car_specs tábla:

Mező	Típus	Leírás
id	bigint, PK	Egyedi azonosító
car_model_id	foreignId → car_models.id	Modell hivatkozás
acceleration	double	0-100 km/h gyorsulás (s)
power	integer	Teljesítmény (hp)
top_speed	integer	Végsebesség (km/h)
weight	integer	Súly (kg)
displacement	integer	Hengerűrtartalom (cm^3)
cylinders	integer	Hengerek száma
transmission	string(255)	Sebességváltó típusa

Color__options, Wheel__options, Interior__options, Accessories táblák:

Mind a négy opciótábla közös szerkezetű: id (PK), car_model_id (FK), name vagy part_name (string), és price (integer). A wheel__options kiegészül color, material, size, img_url mezőkkel; a color__options-nek hex_code mezője van; az interior__options-nek part_name és material/color mezői vannak; az accessories egyszerűbb: csak name és price.

Configs tábla:

Mező	Típus	Leírás
id	bigint, PK	Egyedi azonosító
user_id	foreignId → users.id	Tulajdonos felhasználó
car_model_id	foreignId → car_models.id	Választott modell
color_option_id	foreignId → color__options.id	Választott szín

wheel_option_id	foreignId → wheel__options.id	Választott felni
total_price	integer, nullable	Összár (számolt)
timestamps	created_at, updated_at	Laravel automatikus mezők

Pivot táblák (config_interior_option, config_accessory):

Mindkét pivot tábla közös szerkezetű: id (PK), config_id (FK → configs.id), interior_option_id vagy accessory_id (FK), valamint timestamps. Ezek valósítják meg a több-a-többhöz kapcsolatot a konfigurációk és a belső opciók, illetve a kiegészítők között.

Orders tábla:

Mező	Típus	Leírás
id	bigint, PK, unique	Egyedi rendelési azonosító
user_id	foreignId → users.id	Megrendelő felhasználó
config_id	foreignId → configs.id	Alapul vett konfiguráció
status	enum('pending','completed','cancelled')	Rendelés állapota
message	string(255), nullable	Üzenet / megjegyzés
total_price	integer	Számított összár
timestamps	created_at, updated_at	Laravel automatikus mezők

3.3.3 Kapcsolatok indoklása, ismertetése

A car_models tábla minden konfigurációs entitás központja, mert a választható szín-, felni-, belső- és kiegészítő opciók modellenként eltérnek (pl. egy GT3-hoz más szín-paletta tartozik, mint egy 930 Turbóhoz). Ezért minden opciótábla 1:N kapcsolatban áll a car_models táblával egy idegen kulcson keresztül, kaszkádolt törléssel és frissítéssel. Ha egy modellt törlünk, az ahhoz tartozó opciók is automatikusan eltávolításra kerülnek; az adatkonzisztencia így automatikusan biztosítva van.

A users tábla 1:N kapcsolatban áll a configs és orders táblákkal. Egy felhasználónak több konfigurációja lehet (különböző modellekre, vagy ugyanannak a modellnek különböző variánsai), és ezekből több rendelést is leadhat. A configs és orders között szintén 1:N kapcsolat áll fenn: egy konfiguráció több rendelés alapja is lehet, de egy rendelés mindig pontosan egy konfigurációhoz tartozik.

A configs ↔ interior__options és configs ↔ accessories kapcsolatok több-a-többhöz típusúak, mert egy konfigurációban több belső opciót és több kiegészítőt is választhat a felhasználó, ugyanakkor egy adott opciót/kiegészítőt több konfiguráció is használhat. Ezeket a kapcsolatokat a config_interior_option és a config_accessory pivot tábla valósítja meg, az Eloquent belongsToMany relációval.

A users ↔ configs/orders, valamint a configs ↔ orders kapcsolatok mind kaszkádolt törléssel rendelkeznek: ha egy felhasználó törlődik, a hozzá tartozó konfigurációk és rendelések is automatikusan eltávolításra kerülnek. A jogosultság-érvényesítést a backend Sanctum tokenek abilities alapján végzi: a route guard a managert és admint különálló képességként kezeli, így finomhangolható, hogy melyik műveletet ki végezheti el.

3.4 Komponens/struktúra dokumentáció

3.4.1 A teljes rendszer felépítése

A rendszer felépítése két jól elkülöníthető rétegre oszlik: a Laravel 12 alapú backendre és a Vue 3 alapú frontend felületre. A két réteg között tiszta, REST-alapú kommunikáció valósul meg JSON formátumban, amely biztosítja az adatok konzisztens kezelését és a felhasználói műveletek megbízható kiszolgálását. A backend a Laravel keretrendszer szabványos szerkezetét követi, az autentikáció Laravel Sanctum token-alapú megoldással valósul meg, az autorizáció pedig token abilities (user, manager, admin) rendszerre épül.

3.4.2 A projekt elemei fejlesztés szempontjából

A backend mappastruktúra követi a Laravel 12 alapértelmezett konvencióit: app/Http/Controllers/api a kontrollereknek, app/Http/Requests a Form Request validációs osztályoknak, app/Models az Eloquent modelleknek, database/migrations a sématörténetnek, database/seeders a kezdeti adatoknak, routes/api.php pedig az API útvonalaknak. A frontend szintén logikusan tagolt: src/components a globálisan újrafelhasználható komponenseknek, src/components/layout a layout-keretek számára, src/views/public a nyilvános, src/views/auth a hitelesítési, src/views/app a bejelentkezett felhasználói, src/views/app/admin az admin nézeteknek, src/views/errors a hibaoldalaknak. A src/router/index.js tartalmazza a teljes routerstruktúrát és a beforeEach guardot.

3.4.3 Osztályok

Modellek (Eloquent):

A User modell HasApiTokens trait használatával kezeli a Sanctum tokeneket, és a configs() és orders() metódusokon keresztül publikálja a kapcsolatait. A CarModels modell központi entitás, amelynek hasMany kapcsolatai vannak a colorOptions, wheelOptions, interiorOptions, accessories és configs felé, valamint hasOne kapcsolata a carSpecs felé. A Configs modell tartalmazza a calculateTotalPrice() metódust, amely összegzi a kiválasztott opciók árait, beleértve a pivot táblákon keresztül elérhető interior és accessories listákat is. Az Orders modell belongsTo kapcsolattal kötődik a User-hez és a Configs-hoz; a guarded $total_price mező nem engedi a manuális mass-assignmentet.

Kontrollerek:

Az AuthController kezeli a regisztrációt, bejelentkezést, kijelentkezést, token-ellenőrzést és a felhasználói adatok módosítását. A login művelet token abilities-t generál a felhasználó role-ja alapján: admin → ["user","manager","admin"], manager → ["user","manager"], user → ["user"]. A CarModelController biztosítja a modellek listázását és részletes lekérését (eager loading-gal). A ConfigController az interior és accessories sync() műveleteivel kezeli a több-a-többhöz kapcsolatokat. Az OrderController a /myOrders és /orders végpontok mellett biztosítja a CRUD műveleteket; a manager jogosultság ellenőrzése a ListAll végpontokon és módosítási műveleteken külön kódban történik.

Form Request osztályok:

A Store/Update Form Request osztályok a Laravel ajánlott mintáját követik: az authorize() metódusban a token abilities-t és a tulajdonjogot ellenőrzik (pl. egy user csak saját nevére adhat le rendelést), a rules() metódusban pedig a mezőszintű validációt definiálják. Az UpdateAuthRequest külön gondoskodik arról, hogy a name és email egyediségi vizsgálatából kihagyja a frissítendő rekordot a Rule::unique(...)->ignore(...) hívással.

3.4.4 Kapcsolatok

A modellek közötti kapcsolatok jól illeszkednek az üzleti logikához. A CarModels és a hozzá tartozó opciótáblák között 1:N kapcsolat van, így minden modellhez tetszőlegesen sok színt, felnit, belső opciót és kiegészítőt vehetünk fel. A Configs egyszerre tárolja a felhasználó döntéseit a központi választások (szín, felni) közvetlen idegen kulcsával, és a több elemmel rendelkező döntéseket (interior, accessories) pivot táblán keresztül.

Az Orders modell és a Configs között belongsTo kapcsolat áll fenn, mivel egy rendelés egyetlen konfiguráción alapul. A teljes árat a rendelés mentésekor a backend a config alapján kiszámolja, és így biztosítja, hogy a kliens által küldött adat ne befolyásolhassa az árat. A felhasználó jogosultságát a tokenhez rendelt abilities adja meg: a router guardok és a Form Requestek is ezeket az abilities-t ellenőrzik.

3.5 API dokumentáció

A Porsche Sonderwunsch rendszer REST alapú API-t biztosít az autómodellek, konfigurációk, rendelések és a hitelesítés kezelésére. Az API JSON formátumú kéréseket és válaszokat használ, a védett végpontok eléréséhez Laravel Sanctum token-alapú hitelesítés szükséges. Minden válasz egységes felépítést követ: success boolean, message szöveg, data adatobjektum vagy lista, és opcionális count szám.

Hitelesítési (Auth) végpontok:

Metódus	Útvonal	Leírás	Hitelesítés
POST	/api/auth/register	Új felhasználó regisztrálása	Nem szükséges
POST	/api/auth/login	Bejelentkezés, token generálása	Nem szükséges
POST	/api/auth/logout	Kijelentkezés, aktuális token törlése	Sanctum
GET	/api/auth/check	Token érvényesség és abilities ellenőrzése	Sanctum
GET	/api/auth/allData	Összes felhasználó és rendelés listázása	Sanctum + manager
PATCH	/api/auth/update/{user}	Felhasználó adatainak módosítása	Sanctum (admin v. saját)
GET	/api/user	Bejelentkezett felhasználó lekérdezése	Sanctum

Modellek (Models) végpontok:

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/models	Összes modell listázása	Nem szükséges
GET	/api/models/{model}	Modell részletes adatai	Nem szükséges

		opciókkal	
POST	/api/models	Új modell létrehozása	Sanctum + manager
PUT	/api/models/{model}	Modell módosítása	Sanctum + manager
DELETE	/api/models/{model}	Modell törlése	Sanctum + manager

Konfigurációk (Configs) végpontok:

Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/configs	Összes konfiguráció listázása	Nem szükséges
GET	/api/configs/{config}	Egy konfiguráció részletes adatai	Nem szükséges
GET	/api/myConfigs	Saját konfigurációk lekérdezése	Sanctum
POST	/api/configs	Új konfiguráció létrehozása	Sanctum
PUT	/api/configs/{config}	Konfiguráció módosítása	Sanctum
DELETE	/api/configs/{config}	Konfiguráció törlése	Sanctum

Rendelések (Orders) végpontok:

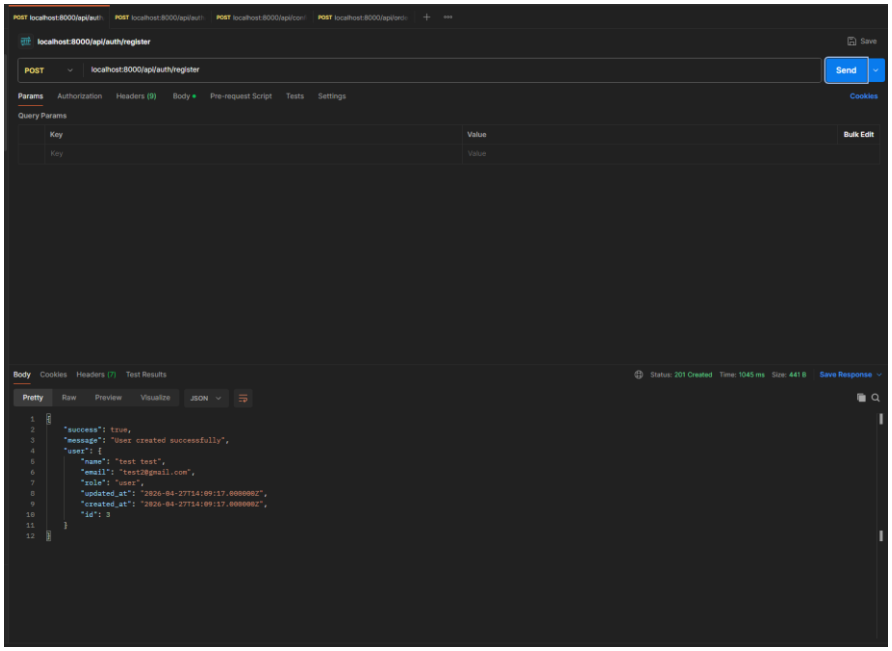
Metódus	Útvonal	Leírás	Hitelesítés
GET	/api/orders	Összes rendelés listázása	Sanctum + manager
GET	/api/orders/{order}	Egy rendelés részletes adatai	Sanctum
GET	/api/myOrders	Saját rendelések lekérdezése	Sanctum
POST	/api/orders	Új rendelés létrehozása (auto-ár)	Sanctum
PUT	/api/orders/{order}	Rendelés módosítása (státusz, üzenet)	Sanctum + manager
DELETE	/api/orders/{order}	Rendelés törlése	Sanctum + manager

Útvonalparaméterek és válasz formátum

Az útvonalakban szereplő {model}, {config}, {order}, {user} paraméterek az adott entitás numerikus azonosítói (példák: /api/models/1, /api/configs/5, /api/orders/12, /api/auth/update/3). Ezeket a Laravel automatikusan modellekhez köti (Route Model Binding). Az API minden esetben JSON formátumú választ ad. Sikeres válasz esetén success: true, message, data és opcionális count mezőket tartalmaz; hiba esetén megfelelő HTTP státuszkódot (403 Forbidden, 404 Not Found, 422 Unprocessable Entity) és az errors objektumot adja vissza.

Példák sikeres válaszokra

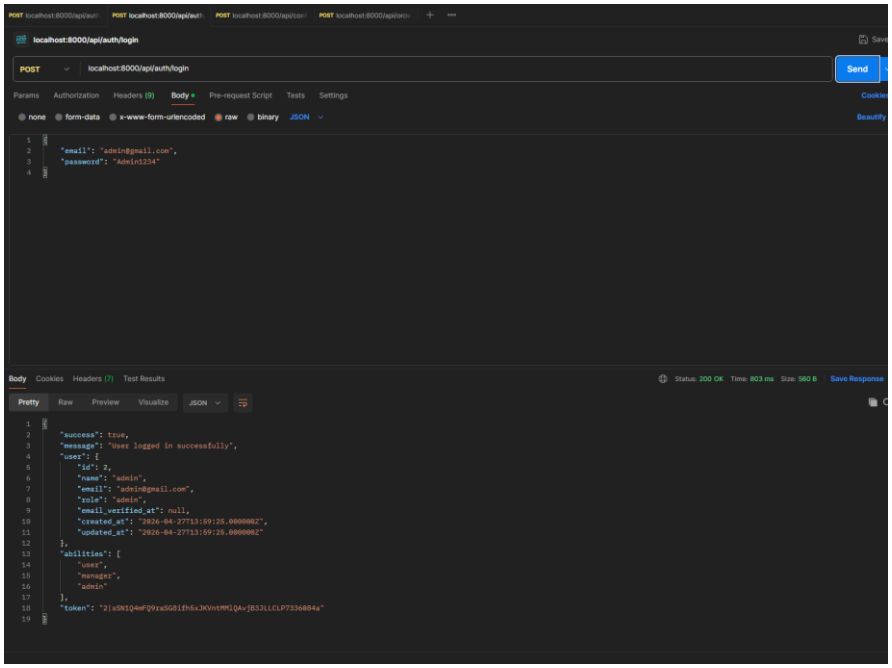
Új felhasználó regisztrálása (POST /api/auth/register):



12. ábra – Postman – új felhasználó regisztrálása

A regisztrációs hívás (12. ábra) sikere esetén a backend egy 201 Created státuszkódot ad vissza, a User objektum publikus mezőivel.

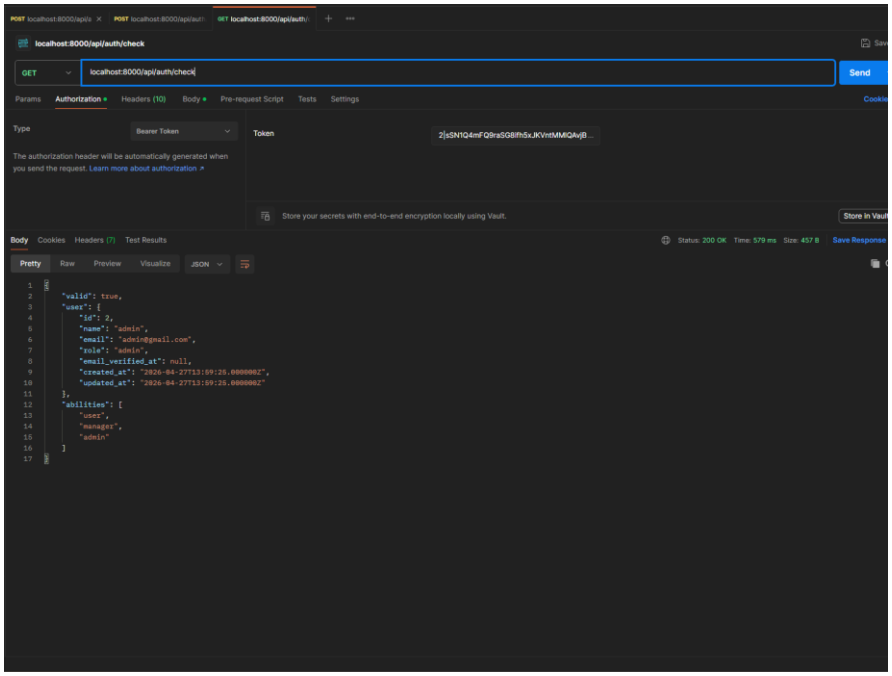
Bejelentkezés (POST /api/auth/login):



13. ábra – Postman – bejelentkezés és token generálás

A bejelentkezés (13. ábra) az email és jelszó alapján hitelesít, és visszaadja a Sanctum tokent, a felhasználói adatokat, valamint az abilities tömbjét.

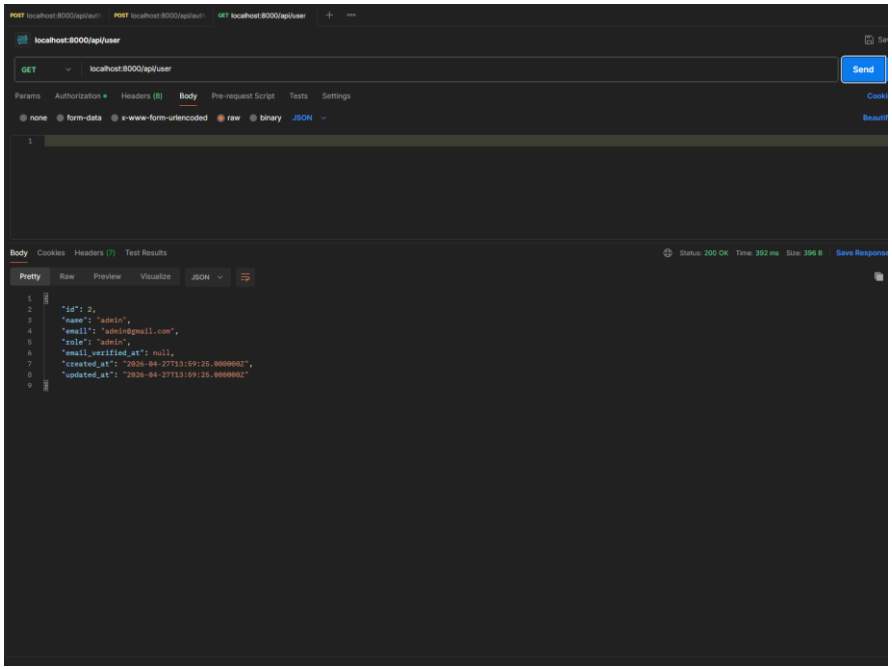
Token érvényesség ellenőrzése (GET /api/auth/check):



14. ábra – Postman – token érvényesség ellenőrzése

Az App.vue gyökérkomponens minden alkalmazás-betöltéskor meghívja ezt a végpontot (14. ábra), hogy frissen tartsa a localStorage-ben tárolt felhasználói adatokat.

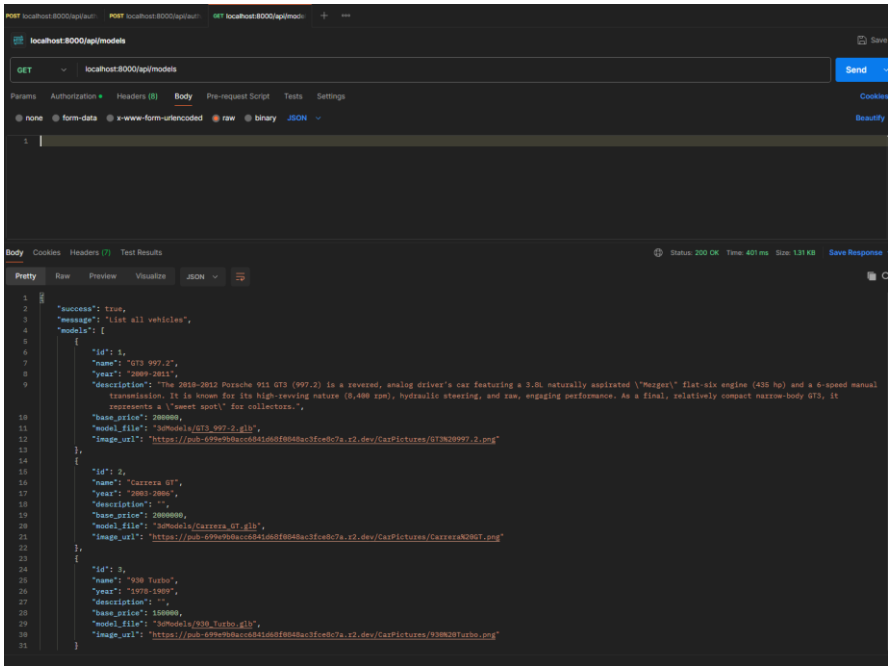
Bejelentkezett felhasználó lekérdezése (GET /api/user):



15. ábra – Postman – bejelentkezett felhasználó adatai

A /api/user a Sanctum middleware által biztosított standard végpont (15. ábra), amely a jelenlegi tokenhez tartozó felhasználót adja vissza.

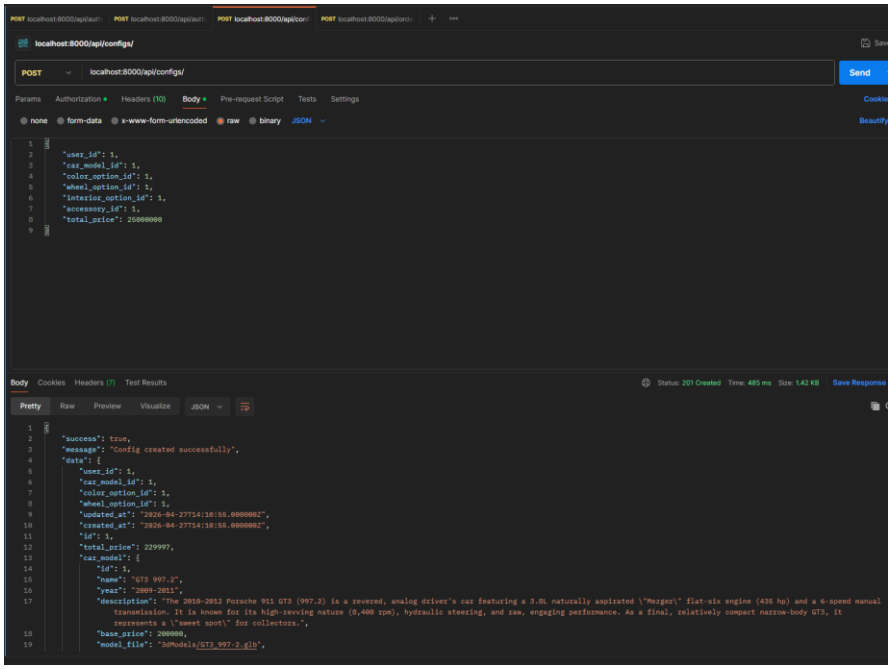
Modellek listázása (GET /api/models):



16. ábra – Postman – összes modell listázása

A modellek lekérdezése (16. ábra) publikus, autentikáció nem szükséges. A válasz tartalmazza az összes elérhető autómodellt és a darabszámot.

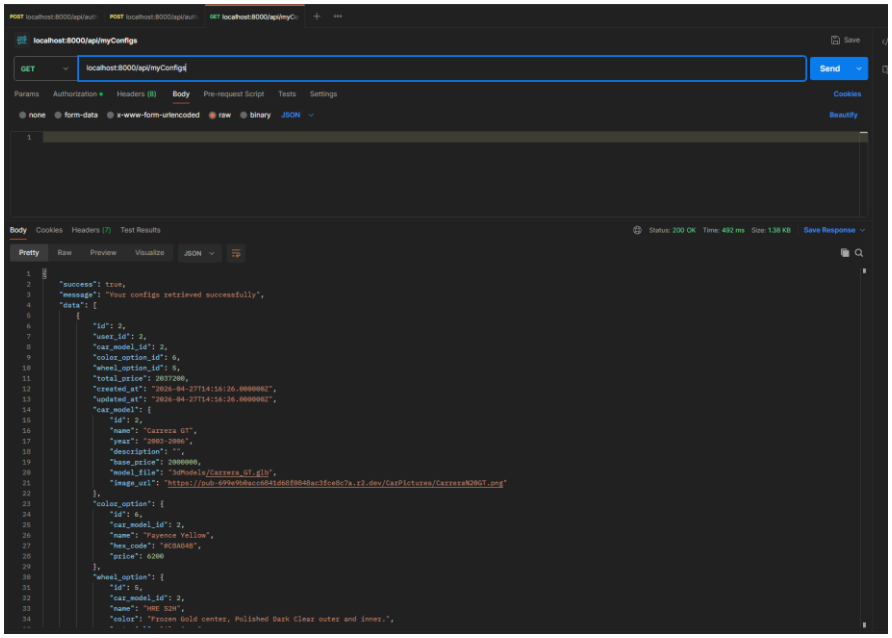
Konfigurációk listázása (GET /api/configs):



17. ábra – Postman – összes konfiguráció listázása

A globális konfiguráció-listázás (17. ábra) demonstrációs / debug célokra publikus.

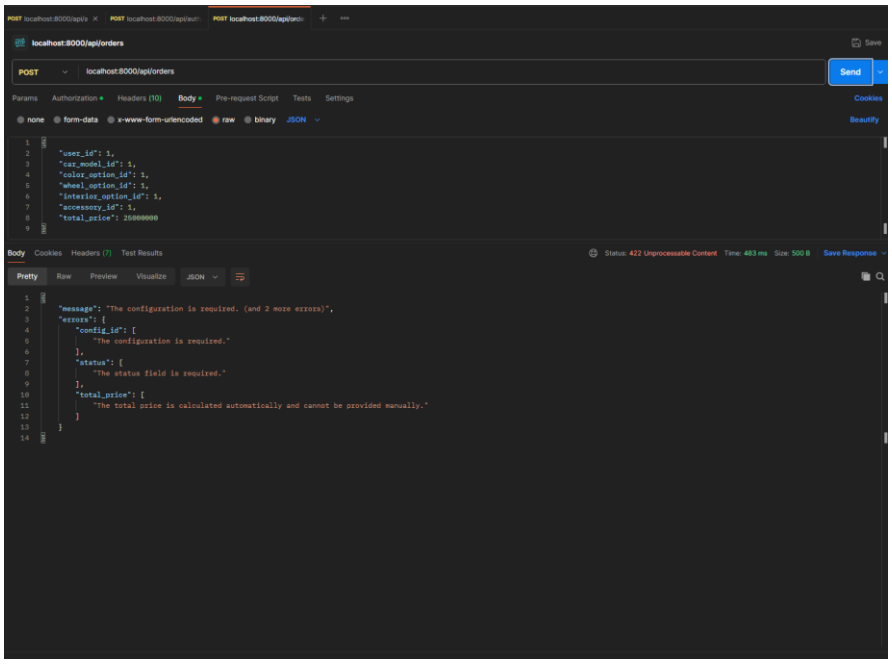
Saját konfigurációk lekérdezése (GET /api/myConfigs):



18. ábra – Postman – bejelentkezett felhasználó konfigurációi

A /api/myConfigs (18. ábra) a Sanctum tokenből kiolvassa a felhasználót, és csak az ő saját konfigurációit adja vissza, eager loadinggal.

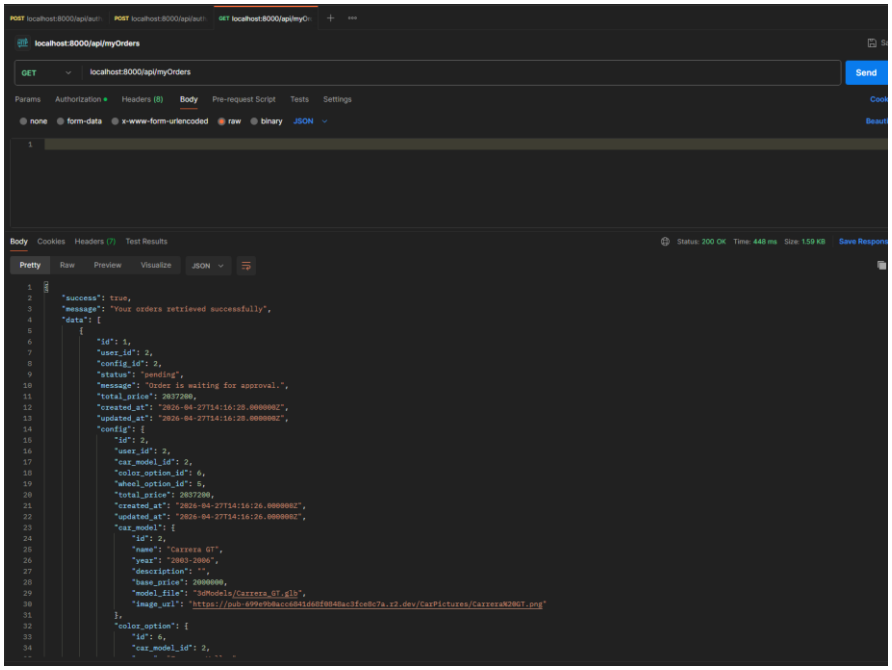
Új rendelés létrehozása (POST /api/orders):



19. ábra – Postman – új rendelés létrehozása

A rendelés POST végpont (19. ábra) a config_id alapján mindig a backend oldalon kalkulált total_price-szal hozza létre a rekordot. Ha a kliens manuálisan próbál árat küldeni, a Form Request prohibited szabálya blokkolja a kérést.

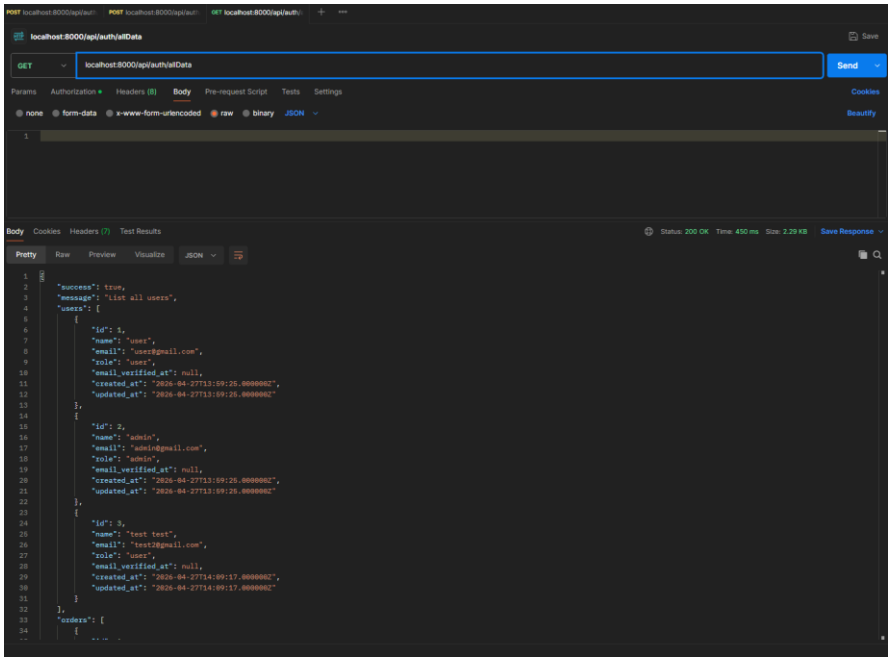
Saját rendelések lekérdezése (GET /api/myOrders):



20. ábra – Postman – bejelentkezett felhasználó rendelései

A /api/myOrders (20. ábra) a felhasználó saját rendeléseit adja vissza, az összes kapcsolódó konfigurációval, modellel, opcióval együtt egy hívásban.

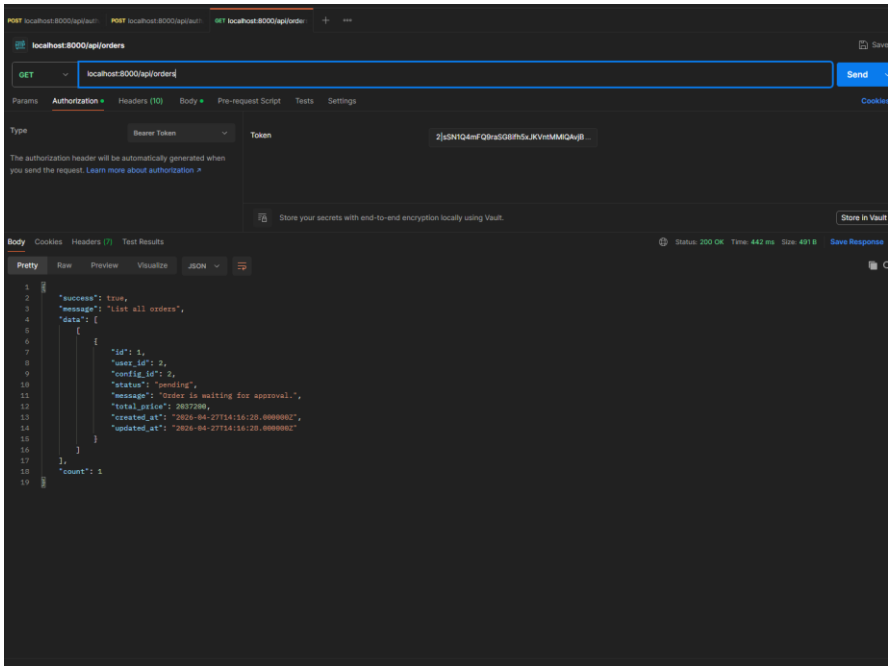
Admin – összes adat lekérése (GET /api/auth/allData):



21. ábra – Postman – admin összes adat lekérése

A manager szintű végpont (21. ábra) az összes regisztrált felhasználót és rendelést adja vissza egyetlen hívásban, kiegészítve a userCount és orderCount mezőkkel.

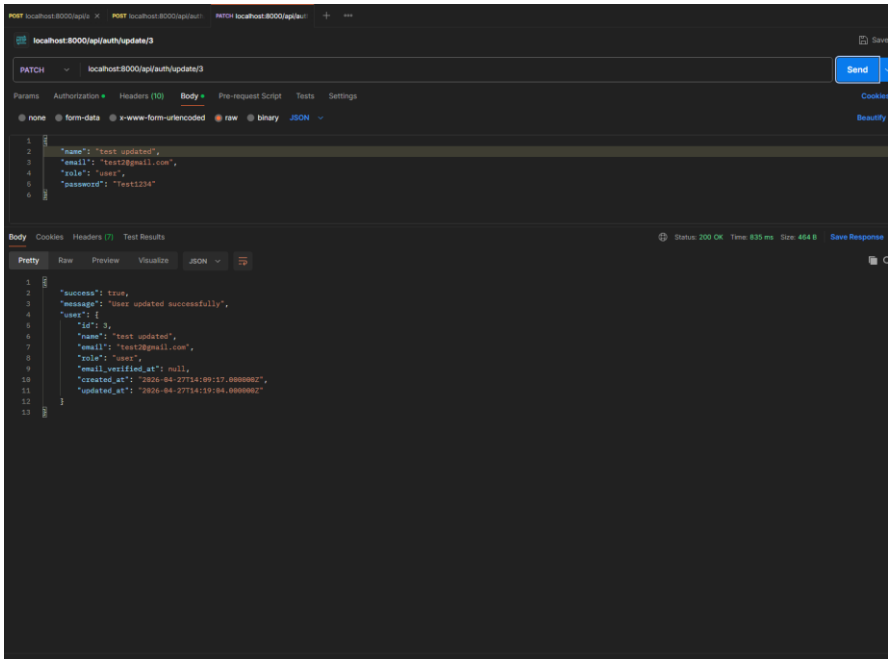
Admin – összes rendelés (GET /api/orders):



22. ábra – Postman – admin összes rendelés

Az /api/orders index (22. ábra) szintén manager jogosultsággal érhető el, és minden rendelést visszaad a rendszerben.

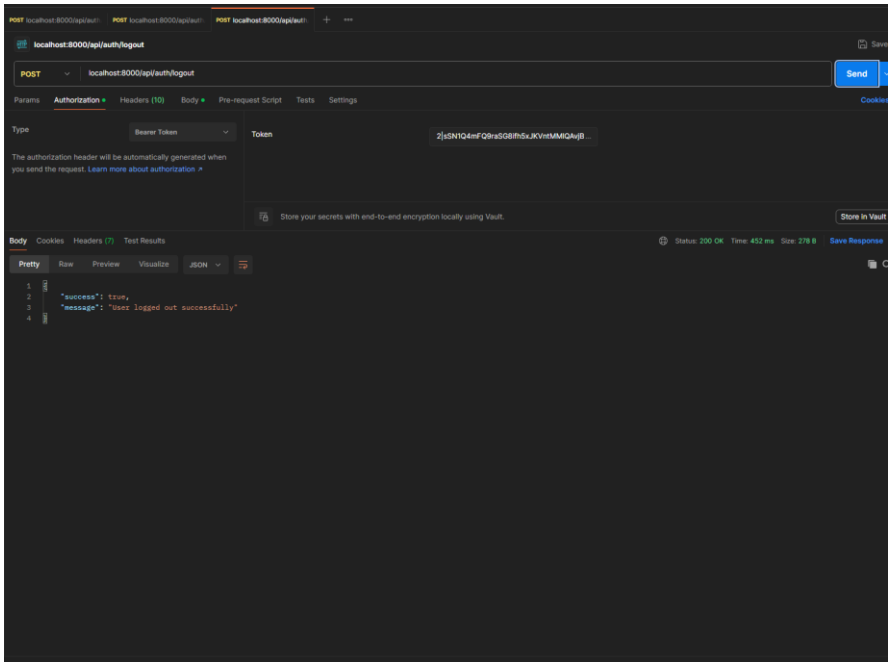
Admin – felhasználó módosítása (PATCH /api/auth/update/{id}):



23. ábra – Postman – admin felhasználó módosítása

Admin tokennel (23. ábra) a Sanctum admin abilities birtokában a felhasználó adatai bárkire vonatkozóan módosíthatók. Egy normál user csak a saját adatait módosíthatja.

Kijelentkezés (POST /api/auth/logout):



24. ábra – Postman – kijelentkezés és token érvénytelenítése

A kijelentkezés (24. ábra) az aktuális tokent törli a personal_access_tokens táblából, így a továbbiakban már nem használható autentikációhoz.

3.6 Teszt dokumentáció

A backend teljes körű tesztelése Postman segítségével történt. A tesztelési folyamat során minden API-végpont külön-külön került ellenőrzésre, beleértve a sikeres és hibás válaszokat, valamint a jogosultságkezelés működését is. A részletes válaszok és a végpontokhoz tartozó példák az előző fejezet képernyőképein láthatóak.

A tesztelés során minden végpont a vártnak megfelelően működött:

- Publikus végpontok (GET /api/models, GET /api/models/{id}, GET /api/configs, GET /api/configs/{id}) minden felhasználó számára elérhetők, és a megfelelő adatokat adják vissza.
- Saját adat végpontok (GET /api/myConfigs, GET /api/myOrders) Sanctum tokennel működnek, és csak a bejelentkezett felhasználó saját adatait adják vissza.
- Manager jogosultsághoz kötött végpontok (GET /api/auth/allData, GET /api/orders index, POST/PUT/DELETE /api/models, PUT/DELETE /api/orders) kizárólag manager vagy admin abilities-ű tokennel érhetők el. User esetén 403 Unauthorized válasz érkezik.
- Hitelesítési végpontok (POST /api/auth/login, POST /api/auth/logout, GET /api/auth/check) a Sanctum tokenalapú azonosítással működnek, és minden esetben a megfelelő státuszkódot és JSON választ adják.

A frontend felület tesztelése a rendszer működésének teljeskörű ellenőrzését szolgálta. A tesztelés célja annak igazolása volt, hogy a felhasználói felület minden funkciója megfelelően működik, a backend API-val való kommunikáció hibamentes, valamint a jogosultságkezelés a grafikus felületen is érvényesül. A tesztelés manuálisan, böngészőben történt, user és admin szerepkörök használatával.

A tesztelés első lépéseként mindkét felhasználó bejelentkezési folyamata ellenőrzésre került. Admin felhasználó esetén a rendszer megjelenítette a profil dropdownban az „Admin Dashboard" linket, a dashboardon pedig a felhasználó- és rendeléskezelő funkciókat. A funkcionális tesztek során ellenőrzésre került a modellek listázása, az egyes modellek adatlapjának megjelenítése, valamint a konfigurátor működése. Az új konfiguráció létrehozása során a kitöltött opciók után az ár valós időben frissült, és a „Save Configuration" megnyomására a rendszer azonnal mentette a backendben, majd átirányított a saját konfigurációk listájára.

A „Place Order" funkció során a frontend elsőként mentette a konfigurációt (ha még nem volt mentve), majd /api/orders POST hívással létrehozott egy „pending" státuszú rendelést. A rendelés azonnal megjelent a felhasználó saját rendelései listájában. Admin oldalon a teszt során az adminisztrátor sikeresen módosította egy felhasználó nevét és jelszavát, illetve egy rendelés státuszát „pending" → „completed"-ra állította. A módosítás után a felhasználó frontendjén is a frissített állapot jelent meg.

User felhasználó esetén ezek a módosítási funkciók nem jelentek meg, így a felület kizárólag megtekintési és saját profil-szerkesztési lehetőséget biztosított, ami igazolja a jogosultságkezelés helyes működését. A védett admin oldalakra való közvetlen URL-megadás esetén a router beforeEach guard 403 Not Authorized oldalra irányította a felhasználót. A 422-es validációs hibákat a frontend helyesen kezelte: a hibás mezők alá beillesztett magyarázattal jelezte azokat. A tesztelés során minden funkció a vártnak megfelelően működött, a felhasználói felület pedig konzisztensen és hibamentesen kommunikált a backend API-val.

4 Továbbfejlesztési lehetőségek

Jelen projektmunka keretében nem valósult meg minden kezdeti ötlet és időközben felmerült fejlesztés, pedig az alkalmazás rejt még néhány érdekes lehetőséget magában. A legfontosabb továbbfejlesztési irány az interaktív 3D autóelőnézet bevezetése. A frontend csomagjai között a Three.js már szerepel, és a konfigurátor felületen ki van alakítva a placeholder a 3D panel számára. A bevezetés során minden modellhez egy .glb fájl tartozik (CarModels.model_file mező), amelyet betölthetünk és a kiválasztott szín- és felni-opciónak megfelelően renderelhetünk valós időben.

A felhasználói élményt jelentősen javítaná egy automatikus e-mail-értesítési funkció, amely a rendelés állapotának változásakor (pending → completed/cancelled) értesítést küld a felhasználónak. Ehhez Laravel oldalon a Mail és a Queue rendszer integrálható, akár Mailgun vagy SES külső szolgáltatóval. Az értesítések tárolása egy notifications tábla révén szintén megoldható lenne.

Az üzleti logika is bővíthető további funkciókkal: kedvezmények és kuponkódok rendszere, szállítási idő- és költségbecslés, részletfizetési opciók megjelenítése. A rendelési folyamatba beépíthető egy fizetési integráció (pl. Stripe vagy Barion) a tényleges ügyletek lebonyolításához. A számlázáshoz külön számlatábla és PDF-generálás (pl. dompdf) is hozzáadható.

A tesztlefedettség növelése szintén előrelépés lenne: PHPUnit feature és unit tesztek bevezetése a kontrollerekre, modellekre, validációs szabályokra és a token abilities ellenőrzésre. A

frontend oldalon a Vitest és a Vue Test Utils használható az egységtesztekhez, valamint a Cypress vagy Playwright a végponti (E2E) tesztekhez.

Az autentikációt erősíteni lehetne kétfaktoros (2FA) bejelentkezéssel, jelszó-visszaállítási funkcióval (a password_reset_tokens tábla már létezik), valamint email-verifikációval. A frontend pedig kibővíthető többnyelvű támogatással (Vue I18n) – jelenleg az interfész angol, de a magyar piacra való lokalizáció hozzáadása viszonylag kis munkával megoldható. A reszponzív viselkedés további finomhangolása szintén tervezhető: a 3D panel megjelenése mobilon külön optimalizációt igényelne.

5 Összegzés

Az általunk választott projektfeladat megvalósítja a vizsgaremek céljául is kitűzött életszerű, jól használható alkalmazást. Igyekeztünk egy olyan programot létrehozni, amelyre akár valós igény is mutatkozhat, és amely egy konkrét, jól körülhatárolt üzleti feladatot old meg: a Porsche „Sonderwunsch" élmény digitális leképezését, ahol az ügyfél részletesen testreszabhatja a vágyott autót, mielőtt rendelést adna le.

A feladatmegosztás nem volt mereven backend-frontend bontású: mindketten dolgoztunk mind a két rétegen, és a 21 mergelt pull request egyértelműen mutatja a közös, iteratív munkafolyamatot. A kezdeti ötletelés, a koncepció megalkotása, az adatmodell kialakítása és az API szerződések rögzítése mind közös munka volt. A megvalósítás során is folyamatosan tudtunk egymástól kérdezni és segíteni egymást. A pull request review-k és Approved jelölések a kódminőség közös biztosításában is fontos szerepet játszottak.

A projekt során elsajátítottuk a Laravel 12 és a Sanctum token-alapú autentikáció használatát, a Vue 3 Options API alapú komponensek és routerek építését, a relációs adatbázis-tervezés legjobb gyakorlatait (idegen kulcs kapcsolatok, kaszkád, pivot táblák), valamint a REST API szerződések tervezését és tesztelését Postman segítségével. A jogosultságkezelés tokenszintű implementációja rugalmas, és bármikor bővíthető további szerepkörökkel.

A Porsche Sonderwunsch konfigurátor mindkettőnknek tetszik, szívesen dolgoznánk vele tovább is, és a feladat megoldásával elégedettek vagyunk. A kód tisztán strukturált, az architektúra átlátható, a továbbfejlesztési lehetőségek pedig egyértelműen kijelölik a következő iterációk irányát.

6 Források

Axios dokumentáció. (2026). Forrás: https://axios-http.com/docs/intro
Bootstrap Icons. (2026). Forrás: https://icons.getbootstrap.com/
Laravel felhasználói dokumentáció. (2025). Forrás: https://laravel.com/docs/12.x
Laravel Sanctum dokumentáció. (2025). Forrás: https://laravel.com/docs/12.x/sanctum
MySQL hivatalos dokumentáció. (2022). Forrás: https://dev.mysql.com/doc/
Postman dokumentáció. (2026). Forrás: https://learning.postman.com/docs/
Three.js dokumentáció. (2025). Forrás: https://threejs.org/docs/
Vite dokumentáció. (2026). Forrás: https://vite.dev/guide/
Vue Router dokumentáció. (2026). Forrás: https://router.vuejs.org/
Vue.js felhasználói dokumentáció. (2026). Forrás: https://vuejs.org/
W3Schools internetes tudástár. (2025). Forrás: https://www.w3schools.com/